

Project Proposal

Title

Rethinking Process Reward Models: Do We Really Need Large Language Models? Exploring Lightweight Reward Function Approximators for Process Reward Modeling

1. Motivation

Process Reward Models (PRMs) have become one of the most successful approaches for evaluating intermediate reasoning steps of Large Language Models. Recent methods such as Math-Shepherd formulate PRM as a step-wise classification problem, while PQM further introduces a theoretically grounded **Q-value ranking objective**, significantly improving verification performance through comparative learning.

Meanwhile, almost every recent PRM follows the same architectural paradigm:

A large Transformer-based language model is used simultaneously as both the language encoder and the reward approximator.

As a consequence, recent PRM research has increasingly relied on larger backbones (from 1B to 7B and beyond), leading to substantial computational costs during both training and inference.

However, this raises a fundamental question that has received little attention:

Does process reward modeling truly require a large Transformer-based reward model, or is the Transformer mainly responsible for semantic understanding while the reward prediction itself can be performed by much lighter neural architectures?

We believe existing PRMs implicitly couple **language understanding** and **reward approximation** into a single model, making it difficult to understand which component actually contributes to the final performance.

2. Research Question

This project investigates the following question:

Can lightweight neural reward function approximators replace Transformer-based reward models while maintaining competitive Process Reward Modeling performance?

Instead of proposing another larger backbone, we aim to understand **what kind of model is actually required for reward prediction once semantic representations are available.**

3. Core Hypothesis

We formulate three hypotheses.

H1. Decoupling Hypothesis

Language understanding and reward approximation are two different problems.

A pretrained language model is primarily responsible for extracting semantic representations of reasoning trajectories, while reward prediction is fundamentally a regression/ranking problem over these representations.

H2. Architectural Hypothesis

Different neural architectures possess different inductive biases for approximating process rewards.

Specifically,

- MLP is suitable for nonlinear pointwise reward approximation.
- CNN captures local structural patterns among hidden representations.
- GRU explicitly models sequential dependencies between reasoning steps.
- Attention Pooling captures global trajectory-level interactions.

These architectural differences may affect how effectively the model learns the Q-value ranking objective.

H3. Efficiency Hypothesis

A frozen lightweight language encoder combined with an expressive neural reward approximator can achieve verification performance comparable to LoRA-based PRMs while requiring significantly fewer trainable parameters, GPU memory, and training time.

4. Proposed Method

Rather than fine-tuning an entire language model as the reward model, we decouple the PRM into two components.

Stage 1: Frozen Semantic Encoder

A small math-oriented language model (e.g., Qwen2.5-0.5B) is frozen and only used to extract hidden representations of each reasoning step.

Question + Reasoning Steps



Frozen Qwen2.5-0.5B

▼
Hidden Representations

The frozen encoder is identical for all experiments.

Stage 2: Lightweight Reward Function Approximator

The hidden representations are then fed into different lightweight neural architectures for Q-value estimation.

We will investigate four reward approximators:

Model	Motivation
Linear	Original PRM reward head (baseline)
MLP	Nonlinear reward approximation
CNN	Efficient local feature extraction
GRU / BiGRU	Sequential dependency modeling
Attention Pooling	Global trajectory aggregation

Unlike previous PRMs, only these lightweight modules are trainable.

Stage 3: Comparative Learning

All architectures are optimized using the **same PQM Comparative Ranking Loss (Eq. 10)**.

Keeping the loss function fixed allows us to isolate the influence of the reward approximator itself.

This enables a controlled comparison between different neural architectures.

5. Experimental Design

Fixed Components

- Frozen Qwen2.5-0.5B encoder
- Same hidden representations
- Same training dataset (Math-Shepherd)
- Same PQM Comparative Loss
- Same optimizer and hyperparameters
- Same evaluation protocol

Compared Methods

Method	Trainable Module	Trainable Params	Loss
Baseline A	LoRA on Qwen2.5-0.5B	~10M	BCE
Baseline B	LoRA on Qwen2.5-0.5B	~10M	PQM
Baseline C	Frozen Encoder + Linear Head	<0.5M	PQM
Ours-MLP	Frozen Encoder + MLP	<1M	PQM
Ours-CNN	Frozen Encoder + CNN	~2M	PQM
Ours-GRU	Frozen Encoder + BiGRU	~3M	PQM
Ours-Attention	Frozen Encoder + Attention Pooling	~2M	PQM

6. Evaluation

Verification Performance

- BON@8
- BON@16
- Step-level Reward Accuracy
- Q-value Ranking Accuracy

Computational Efficiency

- Peak GPU Memory
- Training Time
- Inference Latency
- Number of Trainable Parameters

Representation Analysis

To better understand the learned reward functions, we will additionally analyze

- Q-value distributions
- Correct vs. incorrect step separation
- Feature visualization (e.g., t-SNE/PCA)
- Training convergence

This analysis helps explain **why** certain architectures perform better rather than only reporting accuracy.

7. Expected Contributions

This project aims to provide a systematic study of the architectural assumptions behind Process Reward Models.

Specifically, we expect three contributions.

Contribution 1

We decouple semantic understanding from reward approximation and investigate whether process reward modeling truly requires a Transformer-based reward model.

Contribution 2

We provide the first controlled comparison (within this project scope) of lightweight neural reward function approximators under the same PQM ranking objective, isolating the effect of architecture from language representation.

Contribution 3

We establish an efficient PRM paradigm based on

Frozen Language Encoder + Lightweight Reward Function Approximator

which has the potential to substantially reduce GPU memory, training cost, and deployment complexity while maintaining competitive verification performance.

8. Expected Outcomes

Possible outcomes include:

- Lightweight reward approximators achieve performance close to LoRA-based PRMs while dramatically reducing computational cost.
 - Sequential architectures (GRU or Attention) exploit PQM's ranking supervision more effectively than simpler reward heads.
 - The study provides empirical evidence regarding the role of model architecture in Process Reward Modeling, helping clarify whether future PRM research should focus on larger language backbones or better reward function approximators.
-

9. Impact

Current PRM research primarily improves performance by designing new loss functions or scaling Transformer backbones.

Our project instead asks a more fundamental question:

What is the minimal architecture required for effective Process Reward Modeling?

Answering this question may enable significantly more efficient PRMs for research groups with limited computational resources and provide new insights into the relationship between language representation learning and reward approximation.